

Process Flow Documentation

Walmart Sales Forecasting — Data Science Pipeline Blueprint

A step-by-step technical reference covering data ingestion through model training, evaluation, and interpretability.

Pipeline Overview

A single glance at how raw retail data becomes a validated sales forecast.

This document describes the complete data science pipeline built to forecast Walmart's weekly departmental sales. The pipeline ingests four raw source files, cleans and merges them into a unified analytical dataset, engineers calendar, markdown, and time-series features, trains and compares five forecasting models, and closes with a model interpretability layer using SHAP. Each stage below is documented with the exact transformations applied, the rationale behind them, and the tools/libraries used, so that another data scientist can reproduce the work end-to-end.

1. Data Ingestion
2. Data Cleaning & Transformation
3. Feature Engineering
4. Model Training & Validation
5. Prediction & Post-Processing

High-Level Process Flow Diagram

RAW DATA SOURCES

train.csv | test.csv | stores.csv | features.csv



STAGE 1 — DATA INGESTION

Load with pandas → shape/schema checks → type conversion of `Date` → structural validation (nulls, duplicates, describe())



MERGE LAYER

train + stores (key: Store) → result + features (keys: Store, Date, IsHoliday) → unified DataFrame `df`



STAGE 2 — CLEANING & TRANSFORMATION

Markdown1-5 → fillna(0) | CPI, Unemployment → fillna(median) | IsHoliday → fillna(0), cast int | Type → category codes



STAGE 3 — FEATURE ENGINEERING

Calendar features (Year/Month/Week/Quarter/Season) → Markdown aggregates → Lag & rolling-window sales features → Store/Dept average encodings → One-hot encode Season



SPLIT & SCALE

train_test_split (80/20, random_state=42) → StandardScaler fit on train, applied to test



STAGE 4 — MODEL TRAINING & VALIDATION

Linear Regression

Random Forest (base)

Random Forest (tuned, RandomizedSearchCV)

XGBoost

ARIMA (univariate)



STAGE 5 — PREDICTION & POST-PROCESSING

Predict on hold-out set → compute MAE / RMSE / MAPE / R^2 → residual diagnostics → SHAP interpretability → rank models by RMSE → select best model



OUTPUT

cleaned_walmart_data.csv | model comparison table | feature importance ranking | best-performing model (lowest RMSE)

1 Data Ingestion

Four raw CSV files form the input layer of the pipeline. Each is loaded independently, inspected for shape and schema, and validated before any merging takes place.

1.1 Sources Loaded

File	Contents	Join Key(s)
<code>train.csv</code>	Historical weekly sales by Store & Department	base table
<code>test.csv</code>	Future periods requiring forecasts (no target)	Store, Dept, Date
<code>stores.csv</code>	Store metadata – Type, Size	Store
<code>features.csv</code>	External/economic signals – Temperature, Fuel Price, CPI, Unemployment, Markdown1-5, IsHoliday	Store, Date, IsHoliday

1.2 Loading & Initial Validation

- All four files are read with `pandas.read_csv()` into separate DataFrames.
- Shapes are printed (`train.shape` , `test.shape` , `stores.shape` , `features.shape`) to confirm row/column counts match expectations before proceeding.
- `.head()` is used on each DataFrame to visually sanity-check column names and sample values.
- `.info()` is run on train, stores, and features to confirm dtypes and identify columns that will need casting.
- `.isnull().sum()` is run on all three to produce an initial missing-value inventory.
- `.describe()` is run on train and features to check numeric ranges for obvious anomalies (e.g. negative sales).
- `.duplicated().sum()` is run on all three tables to confirm there are no exact duplicate rows.

1.3 Date Standardization

The `Date` column in `train` , `test` , and `features` is converted from string to `datetime64` using `pd.to_datetime()` . This is done **before** merging so that Date can be used reliably as a join key and later for calendar feature extraction.

1.4 Merging into a Unified Dataset

Two sequential left-joins assemble the single working DataFrame `df` :

1. **Merge 1:** `train` LEFT JOIN `stores` on `Store` — attaches store `Type` and `Size` to every sales record.
2. **Merge 2:** result LEFT JOIN `features` on `[Store, Date, IsHoliday]` — attaches economic and markdown signals for the matching store/week.

Rationale: A left join (rather than inner) is used at both steps so that no sales records are dropped even if a corresponding store or feature row is temporarily missing — those gaps are handled explicitly in the cleaning stage rather than silently discarding data.

The merged shape is printed and `.head()` / `.tail()` are displayed to confirm the join behaved as expected across the full date range.

2 Data Cleaning & Transformation

2.1 Missing Value Audit

After merging, a missing-value report is rebuilt as a DataFrame combining raw count and percentage (`Missing Values`, `Percentage`), sorted descending, so the columns needing the most attention are immediately visible. This mirrors the audit done in Stage 1 but now reflects gaps introduced by the merge (e.g. features not available for every store/week).

2.2 Missing Value Strategy

Column(s)	Strategy	Reasoning
MarkDown1– MarkDown5	<code>fillna(0)</code>	A missing markdown value means no promotional markdown was active that week — 0 is the semantically correct value, not an unknown.
CPI	<code>fillna(median)</code>	Median is robust to outliers/skew in macro-economic indices and preserves the overall distribution better than mean imputation.
Unemployment	<code>fillna(median)</code>	Same reasoning as CPI — a slow-moving macro indicator where median imputation avoids distorting extreme-year values.
IsHoliday	<code>fillna(0)</code> , cast to <code>int</code>	Absence of a holiday flag from the features table is treated as "not a holiday"; casting to int makes it model-ready.

Verification: `df.isnull().sum()` is re-run immediately after imputation to confirm zero remaining nulls in the treated columns before feature engineering begins.

2.3 Categorical Encoding

- `Type` (store type A/B/C) is cast to a pandas `category` dtype, then encoded into `Type_Code` via `.cat.codes` — a compact integer encoding suitable for tree-based models.
- The original `Type` column is retained temporarily for EDA (boxplots of sales by type) and is dropped later, just before the train/test split, once `Type_Code` has taken over its modeling role.

2.4 Outlier Handling

Outliers are not removed from the dataset. Instead, the pipeline treats extreme values (e.g. very large weekly sales during holiday spikes) as legitimate signal, since the forecasting objective must anticipate exactly those spikes. Outlier behavior is instead assessed visually — a histogram/KDE of `Weekly_Sales`, a boxplot of sales by `Type`, and a residual-vs-predicted scatter plot after modeling — so that any distortion they cause in later model errors can be diagnosed rather than masked.

2.5 Checkpoint Output

The cleaned, merged DataFrame is persisted to `cleaned_walmart_data.csv` via `df.to_csv(index=False)`. This checkpoint allows the cleaning stage to be re-run independently of the modeling stages downstream.

3 Feature Engineering Pipeline

Every engineered feature below is derived from the cleaned dataset. Each is listed with its construction logic and the rationale for including it in the model.

3.1 Calendar Features

Feature	Construction	Rationale
Year, Month, Day	<code>Date.dt.year / .month / .day</code>	Captures broad seasonal and yearly trend effects in sales.
Week	<code>Date.dt.isocalendar().week</code>	Retail sales are strongly week-of-year cyclical (e.g. Thanksgiving, Christmas weeks).
Quarter	<code>Date.dt.quarter</code>	Coarser seasonal grouping useful for quarter-level trend patterns.
DayOfWeek / IsWeekend	<code>Date.dt.dayofweek</code> , then <code>>=5</code>	Retained for completeness of the calendar signal, even though weekly-level sales limits its direct effect.
Season	Mapped from Month via a custom function (Winter/Spring/Summer/Autumn)	Groups months into a lower-cardinality seasonal signal that is easier for linear models to exploit; later one-hot encoded.

3.2 Markdown / Promotion Features

Feature	Construction	Rationale
Total_MarkDown	Sum of Markdown1–5	Single aggregate signal for total promotional depth in a given week.
Average_MarkDown	Row-wise mean of Markdown1–5	Normalizes promotional intensity independent of how many markdown types were active.
Active_MarkDowns	Count of Markdown columns > 0	Captures breadth of promotion (how many concurrent markdown types), a signal distinct from total value.

3.3 Time-Series (Lag & Rolling) Features

Before constructing lag features, the DataFrame is explicitly sorted by `[Store, Dept, Date]` to guarantee chronological order within every store-department group — a mandatory precondition for `shift()` and `rolling()` operations to be correct.

Feature	Construction	Rationale
Lag_1, Lag_2, Lag_4	<code>groupby([Store, Dept]) [Weekly_Sales].shift(1/2/4)</code>	Sales are highly autocorrelated; prior week(s)' sales are among the strongest predictors of the current week.
Rolling_Mean_4	<code>shift(1).rolling(4).mean()</code> per group	Smooths short-term noise, giving the model a recent-trend baseline (shifted to avoid leakage).
Rolling_STD_4	<code>shift(1).rolling(4).std()</code> per group	Captures recent sales volatility per store-department, useful where variance itself is predictive.
Holiday_Next_Week	<code>groupby([Store, Dept]) [IsHoliday].shift(-1)</code>	Captures pre-holiday demand build-up (e.g. week before Thanksgiving) as a forward-looking signal.
Store_Avg_Sales	Mean Weekly_Sales mapped by Store	Encodes store-level baseline scale (a large store trends higher regardless of week).
Dept_Avg_Sales	Mean Weekly_Sales mapped by Dept	Encodes department-level baseline scale (e.g. groceries vs. electronics).

Leakage control: All lag/rolling windows use `shift(1)` before rolling, ensuring the current week's own sales value is never used to construct its own feature. Remaining NaNs produced at the start of each store-department series (where no prior history exists) are filled with 0.

3.4 Encoding for Modeling

- `Season` is one-hot encoded (`pd.get_dummies(..., drop_first=True)`) — performed **before** the X/y split so train and test share identical dummy columns.
- `Date` and the original string `Type` column are dropped from the modeling frame (`df_model`) once their derived numeric equivalents — calendar parts and `Type_Code` — are in place.
- A sanity check confirms zero remaining `object / category` columns in `X_train` before modeling.

4 Model Training & Validation

4.1 Train/Test Split Strategy

`Weekly_Sales` is separated as target `y`; all remaining engineered columns form `X`. The data is split using `train_test_split(test_size=0.20, random_state=42, shuffle=True)`, producing an 80/20 train/test partition. `random_state=42` is fixed throughout for reproducibility.

Note on shuffling: The regression models (Linear Regression, Random Forest, XGBoost) use a shuffled random split, since they treat each row as an independent store-department-week observation enriched with lag/rolling features that already encode temporal order. The ARIMA model, by contrast, uses a strict chronological 80/20 split (no shuffling) on the aggregated weekly sales series, since ARIMA depends entirely on preserving time order.

4.2 Feature Scaling

`StandardScaler` is fit exclusively on `X_train` and then applied to transform both `X_train` and `X_test`, preventing information from the test set leaking into the scaling parameters (mean/variance).

4.3 Models Trained

Model	Configuration	Purpose in Pipeline
Linear Regression	Default parameters	Interpretable linear baseline.
Random Forest (base)	<code>n_estimators=10</code> , <code>random_state=42</code>	Quick non-linear baseline before tuning.
Random Forest (tuned)	<code>RandomizedSearchCV</code> over <code>n_estimators</code> , <code>max_depth</code> , <code>min_samples_split</code> , <code>min_samples_leaf</code> , <code>max_features</code> ; <code>cv=3</code> , <code>n_iter=2</code> , scoring= <code>neg_mean_absolute_error</code>	Hyperparameter-optimized ensemble model.
XGBoost	<code>n_estimators=300</code> , <code>learning_rate=0.05</code> , <code>max_depth=8</code> , <code>objective=reg:squarederror</code>	Gradient-boosted trees for higher predictive accuracy.
ARIMA	Order <code>(5,1,0)</code> on store-aggregated weekly sales series	Classical univariate time-series benchmark.

4.4 Cross-Validation Approach

Hyperparameter search for Random Forest uses 3-fold cross-validation (`cv=3`) within `RandomizedSearchCV`, scored on negative MAE, sampling 2 random parameter combinations from the defined grid. `TimeSeriesSplit` is imported and available in the pipeline for time-aware validation, reflecting the temporal nature of the data, alongside the standard random-split evaluation used for the tree/linear models.

4.5 Training Workflow Summary

1. Fit each model on `X_train` / `y_train` (scaled features for Linear Regression's underlying assumptions; raw features accepted natively by tree-based models).
2. Generate predictions on the held-out `X_test`.
3. Compute MAE, RMSE, and R^2 per model immediately after training for early comparison.
4. For Random Forest, repeat with the `RandomizedSearchCV`-selected best estimator, refit on the full training set.
5. Fit ARIMA separately on the chronologically split aggregate sales series.

5 Prediction & Post-Processing

5.1 Prediction Generation

Each trained model produces point predictions on its respective hold-out set: `lr_pred`, `rf_pred`, `best_rf_pred`, `xgb_pred` on `X_test`, and `arima_pred` via `.forecast(steps=len(test_series))` on the chronological test window.

5.2 Evaluation Metrics

Metric	Purpose
MAE (Mean Absolute Error)	Average magnitude of prediction error in original sales units — easy to interpret business-wise.
RMSE (Root Mean Squared Error)	Penalizes larger errors more heavily; used as the primary metric for ranking and selecting the best model.
MAPE (Mean Absolute Percentage Error)	Scale-independent error, useful for comparing performance across stores/departments of very different sales volume.
R ² Score	Proportion of variance in Weekly_Sales explained by the model.

All metrics are consolidated into a single results DataFrame per model and sorted by RMSE ascending to identify the best performer at a glance.

5.3 Residual Diagnostics (Post-Processing)

- **Actual vs. Predicted scatter plot** (XGBoost) with a 45° reference line — visually assesses systematic over/under-prediction.
- **Residual plot** (predicted vs. residuals) with a zero reference line — checks for heteroscedasticity or non-random error patterns.
- **Residual distribution histogram/KDE** — checks whether errors are approximately centered and normally distributed, validating model assumptions.

5.4 Model Interpretability (SHAP)

SHAP's `TreeExplainer` is applied to the trained XGBoost model over `X_test` to compute per-feature SHAP values. Two summary visualizations are produced:

- A beeswarm summary plot showing the direction and magnitude of each feature's impact on individual predictions.
- A bar-style summary plot ranking features by mean absolute SHAP value (global importance).

This is cross-checked against XGBoost's native `feature_importances_`, sorted descending and visualized as a top-10 bar chart, giving two independent views of which engineered features (e.g. lag sales, store/department averages) drive the forecast most.

5.5 Final Model Selection

The consolidated results table is sorted by RMSE ascending; the top row (`results.sort_values("RMSE").iloc[0]`) is programmatically selected as the best-performing model for deployment/reporting purposes.

Reproducibility note: All random operations across the pipeline (train/test split, Random Forest, RandomizedSearchCV, XGBoost) use `random_state=42`, so re-running the pipeline against the same source CSVs will reproduce identical splits, model fits, and metrics.